

CHAPTER 04

OpenCV 주요 기능

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

4장 OpenCV 주요 기능

4.1 카메라와 동영상 파일 다루기

4.2 다양한 그리기 함수

4.3 이벤트 처리

4.4 OpenCV 데이터 파일 입출력

4.5 유용한 OpenCV 기능

4.3 이벤트 처리

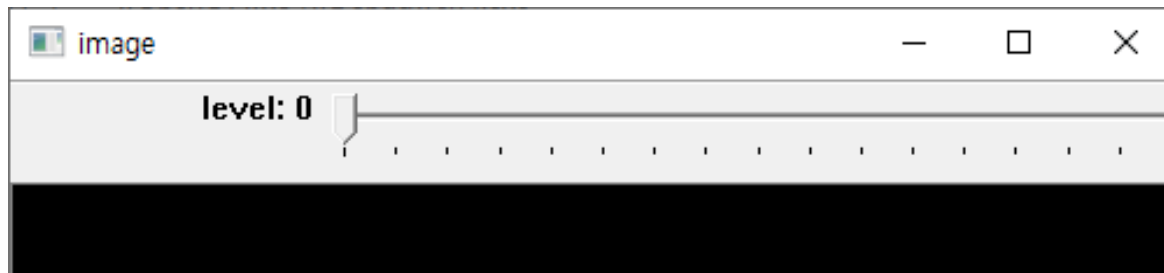
이벤트 처리

- 이벤트(event) : 마우스, 키보드 등 하드웨어를 동작 시켰을 때 각 장치에서 운영체제에 알리는 신호
- 이벤트 핸들러(event handler) : 이벤트가 발생했을 때 처리해야 할 일을 작성해놓은 함수, 운영체제에 의해 자동으로 호출됨(콜백 함수)



트랙바 사용하기

- 영상 출력 창에 부착되어 프로그램 동작 중에 사용자가 지정된 범위 안의 값을 선택할 수 있게 해주는 GUI(Graphic User Interface) 기능 중의 하나
- OpenCV는 Windows, Linux, Mac OS 운영 체제에서 공통으로 사용할 수 있는 트랙바(trackbar) 인터페이스를 제공함



트랙바 사용하기

- 트랙바는 사용자가 지정한 영상 출력 창의 상단에 부착
- 트랙바에는 고유한 이름을 지정해야 하며, 이름은 트랙바 왼쪽에 나타남
- 트랙바의 위치는 사용자가 마우스를 이용하여 이동시킬 수 있고, 트랙바의 현재 위치는 트랙바 이름 옆에 함께 표시됨
- 트랙바가 가리킬 수 있는 최대 위치는 트랙바 생성 시 지정할 수 있으며, 최소 위치는 항상 0으로 고정되어 있음

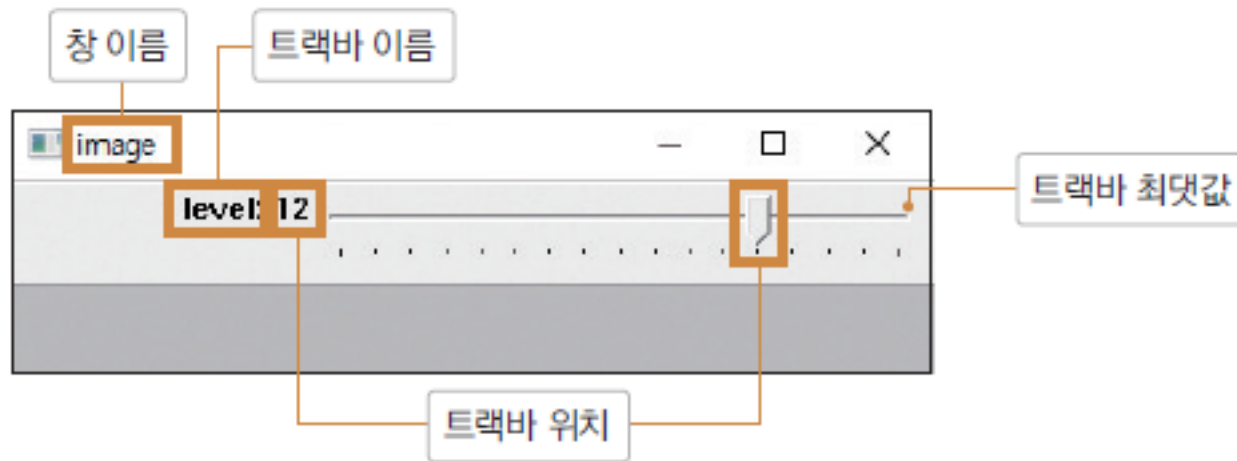


그림 4-9 트랙바 인터페이스

트랙바 사용하기

```
int createTrackbar(const String& trackbarname, const String& winname,
                  int* value, int count, TrackbarCallback onChange = 0,
                  void* userdata = 0);
```

- **trackbarname** 트랙바 이름
- **winname** 트랙바를 생성할 창 이름
- **value** 트랙바 위치를 받을 정수형 변수의 주소
- **count** 트랙바 최대 위치
- **onChange** 트랙바 위치가 변경될 때마다 호출되게 만들 콜백 함수 이름(함수의 포인터)
만약 NULL을 지정하면 콜백 함수는 호출되지 않고 value로 지정한 변수 값만 갱신됩니다.
- **userdata** 트랙바 콜백 함수에 전달할 사용자 데이터의 포인터
- **반환값** 정상 동작하면 1을, 실패하면 0을 반환합니다.

트랙바 사용하기

- 지정된 윈도우에 트랙바를 생성함
- 콜백 함수의 자료형 TrackbarCallback은 다음과 같이 재정의됨

```
typedef void (*TrackbarCallback)(int pos, void* userdata);
```

```
-> typedef void (*      )(int pos, void* userdata) TrackbarCallback
```

- 트랙바 콜백 함수 이름을 함수 포인터 onChange에 전달해야 함

트랙바 사용하기

```
void trackbar_handler(int pos, void* userdata)
{
    // 내용은 마음대로 작성 가능
}
```

- 트랙바 콜백 함수(이벤트 핸들러)는 트랙바 위치가 변경될 때 이벤트가 발생하고 운영체제에 의해 호출되는 함수임
- 트랙바 콜백 함수 이름은 사용자 마음대로 지정해도 되지만, 함수의 매개변수 목록과 반환형은 반드시 정해진 형태를 따라야 함
- **pos** : 현재 트랙바의 위치 정보가 전달됨
- **userdata** : createTrackbar() 함수에서 지정한 사용자 데이터의 주소값이 전달됨

트랙바 사용하기

```
int getTrackbarPos(const String& trackbarname, const String& winname);
```

- `trackbarname` 트랙바 이름
- `winname` 트랙바가 부착되어 있는 창 이름
- 반환값 지정한 트랙바의 현재 위치

```
void setTrackbarPos(const String& trackbarname, const String& winname, int pos);
```

- `trackbarname` 트랙바 이름
- `winname` 트랙바가 부착되어 있는 창 이름
- `pos` 트랙바를 이동할 위치

- 트랙바를 생성한 후, 트랙바의 현재 위치를 알고 싶다면 `getTrackbarPos()` 함수를 사용
- 프로그램 동작 중 트랙바 위치를 강제로 특정 위치로 옮기고 싶다면 `setTrackbarPos()` 함수를 사용

예제1: 트랙바 위치 출력하기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
void on_change(int pos, void* userdata);
int main(void)
{
    Mat src = imread("lenna.bmp");
    if (img.empty()) { cerr << "Image load failed!" << endl; return -1;}
    namedWindow("src"); // 윈도우를 먼저 만들어야 함
    createTrackbar("level", "src", 0, 16, on_change);
    imshow("src", src);
    waitKey();
    return 0;
}
void on_change(int pos, void* userdata)
{
    static int i = 0;
    cout << "트랙바 이벤트 횟수:" << ++i << endl;
    cout << "트랙바 위치:" << pos << endl;
}
```

예제1: 트랙바 위치 출력하기

- 트랙바 이벤트 발생시 실행 흐름을 이해하는 것이 매우 중요

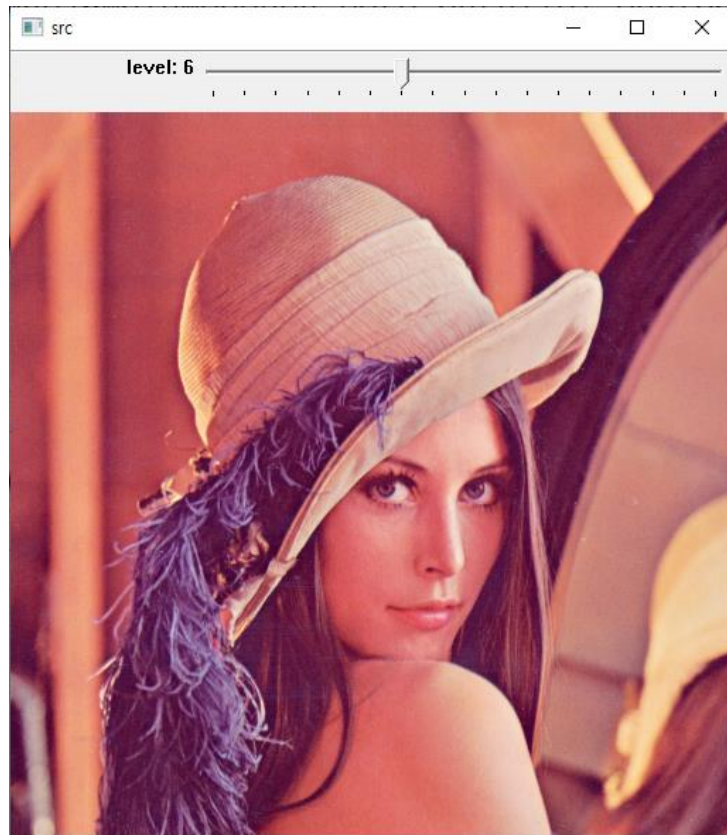
```
// 메인함수
int main(void)
{
    ...
    namedWindow("src"); // 윈도우를 먼저 만들어야 함
    createTrackbar("level", "src", 0, 16, on_change);
    imshow("src", src);
    waitKey();
    return 0;
}
```

트랙바 이벤트 발생시

이벤트 핸들러 종료시

```
// 이벤트 핸들러
void on_change(int pos, void* userdata)
{
    ...
}
```

예제1: 트랙바 위치 출력하기



```
D:\Users\2sungryul\Dropbox\Lecture\2021년2...
트랙바 이벤트 횟수:1
트랙바 위치:1
트랙바 이벤트 횟수:2
트랙바 위치:2
트랙바 이벤트 횟수:3
트랙바 위치:3
트랙바 이벤트 횟수:4
트랙바 위치:4
트랙바 이벤트 횟수:5
트랙바 위치:5
트랙바 이벤트 횟수:6
트랙바 위치:6
```

코드4-12 : 트랙바 사용하기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
Mat img(400, 400, CV_8UC1, Scalar(0));
void on_level_change(int pos, void* userdata);
int main(void)
{
    namedWindow("image");
    createTrackbar("level", "image", 0, 16, on_level_change);
    int key;
    while (true) {
        imshow("image", img);
        key = waitKey(10);
        if (key == 'q') break;
    }
    return 0;
}
void on_level_change(int pos, void* userdata)
{
    img.setTo(pos * 16); // img = Scalar(pos * 16)
}
```

코드4-12 : 트랙바 사용하기

- 초기에는 검정색 배경이고 사용자가 마우스를 이용하여 트랙바 위치를 조정하면 그에 해당하는 밝기로 배경이 변경됨

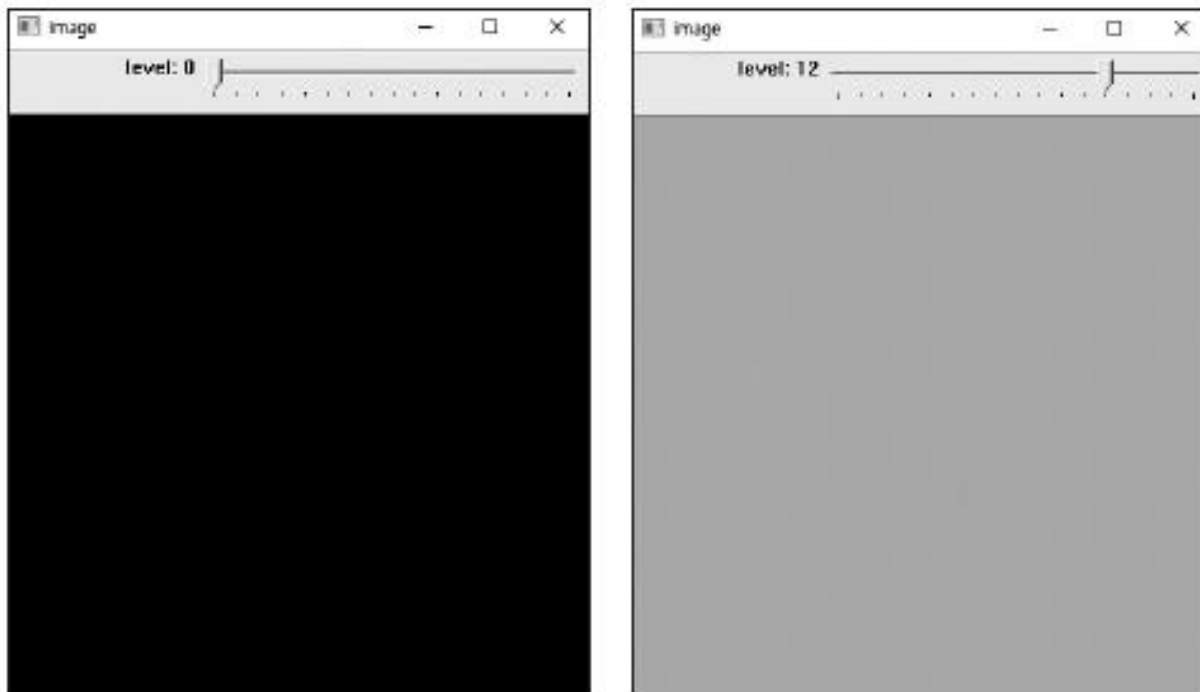


그림 4-10 트랙바를 이용하여 그레이스케일 레벨 표현하기

2개 이상의 함수에서 변수 공유하기

- 방법1: 공유할 변수를 전역변수로 선언하고 모든 함수에서 사용
- 방법2: 공유할 변수를 **main함수에 지역변수로 선언하고** 그것의 주소를 이벤트 핸들러의 매개 변수(**void 포인터**)로 전달하는 방법(초기화함수에서 인수로 전달 -> 이벤트핸들러 호출시 운영체제에 의해 전달됨)

예제2: void 포인터 사용

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
void on_level_change(int pos, void* userdata);
int main(void)
{
    Mat img(400, 400, CV_8UC1, Scalar(0));
    namedWindow("image");
    createTrackbar("level", "image", 0, 16, on_level_change, (void*)&img);
    int key;
    while (true) {
        imshow("image", img);
        key = waitKey(10);
        if (key == 'q') break;
    }
    return 0;
}
void on_level_change(int pos, void* userdata)
{
    ((Mat*)userdata)->setTo(pos * 16);
}
```

예제3: 전역변수 이용하기

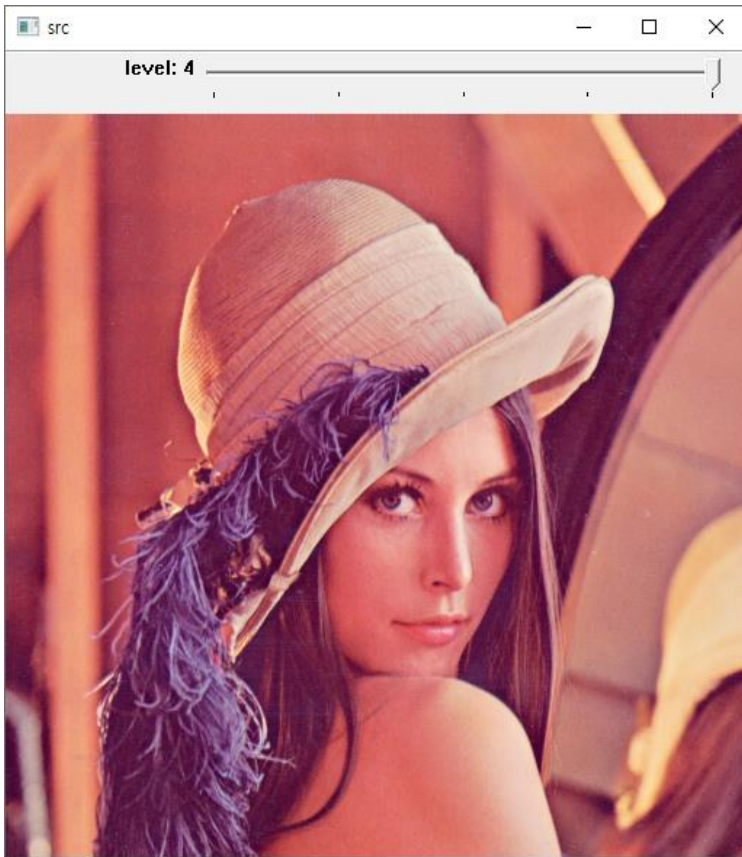
```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
void on_mouse(int event, int x, int y, int flags, void* userdata);
void on_trackbar(int pos, void* userdata);
int track_pos;           // 공유할 변수를 전역변수로 선언
int main(void)
{
    Mat src = imread("lenna.bmp");
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}
    namedWindow("src");
    setMouseCallback("src", on_mouse);
    createTrackbar("level", "src", 0, 4, on_trackbar);
    imshow("src", src);
    waitKey();
    return 0;
}
```

예제3: 전역변수 이용하기

```
void on_mouse(int event, int x, int y, int flags, void* userdata)
{
    switch (event) {
    case EVENT_LBUTTONDOWN:
        cout << "트랙바 위치:" << track_pos << endl; // 공유변수 출력
        break;
    default:
        break;
    }
}

void on_trackbar(int pos, void* userdata)
{
    track_pos = pos; // 공유변수에 위치 저장
}
```

예제3: 전역변수 이용하기



```
D:\Users\2sungryul\Dropbox>
트랙바 위치:0
트랙바 위치:1
트랙바 위치:2
트랙바 위치:3
트랙바 위치:4
```

예제4: void포인터 이용

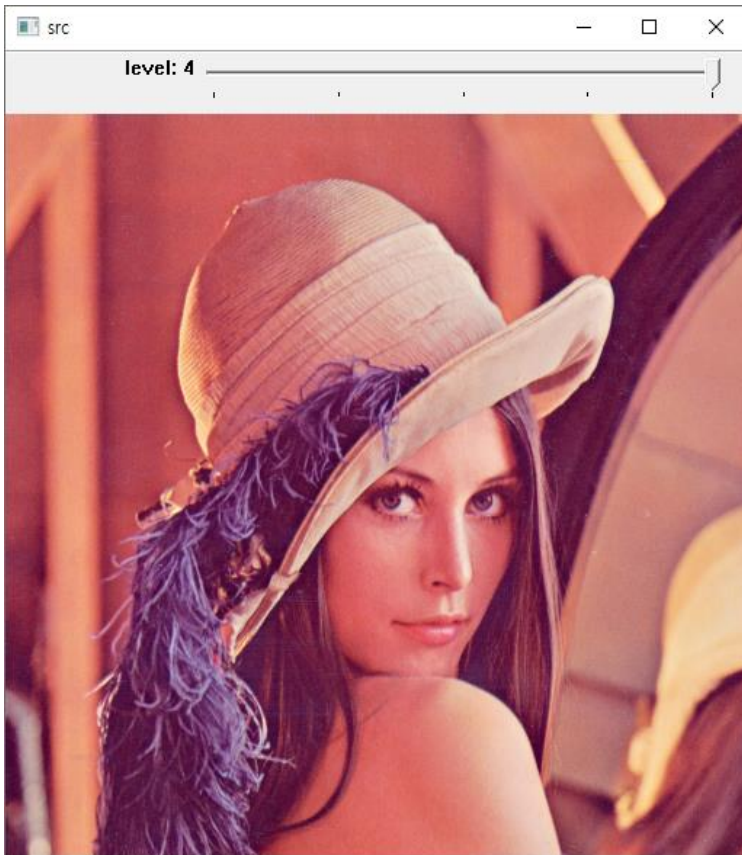
```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
void on_mouse(int event, int x, int y, int flags, void* userdata);
void on_trackbar(int pos, void* userdata);
int main(void)
{
    int track_pos = 0;           // 공유할 변수를 지역변수로 선언
    Mat src = imread("lenna.bmp");
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1;}
    namedWindow("src");
    setMouseCallback("src", on_mouse, &track_pos); // 공유변수주소전달
    createTrackbar("level", "src", 0, 4, on_trackbar, &track_pos); //공유변수주소전달
    imshow("src", src);
    waitKey();
    return 0;
}
```

예제4: void포인터 이용

```
void on_mouse(int event, int x, int y, int flags, void* userdata)
{
    switch (event) {
        case EVENT_LBUTTONDOWN:
            cout << "트랙바 위치:" << *(int*)userdata << endl; // 공유변수 출력
            break;
        default:
            break;
    }
}

void on_trackbar(int pos, void* userdata)
{
    *(int*)userdata = pos; // 공유변수에 위치 저장
}
```

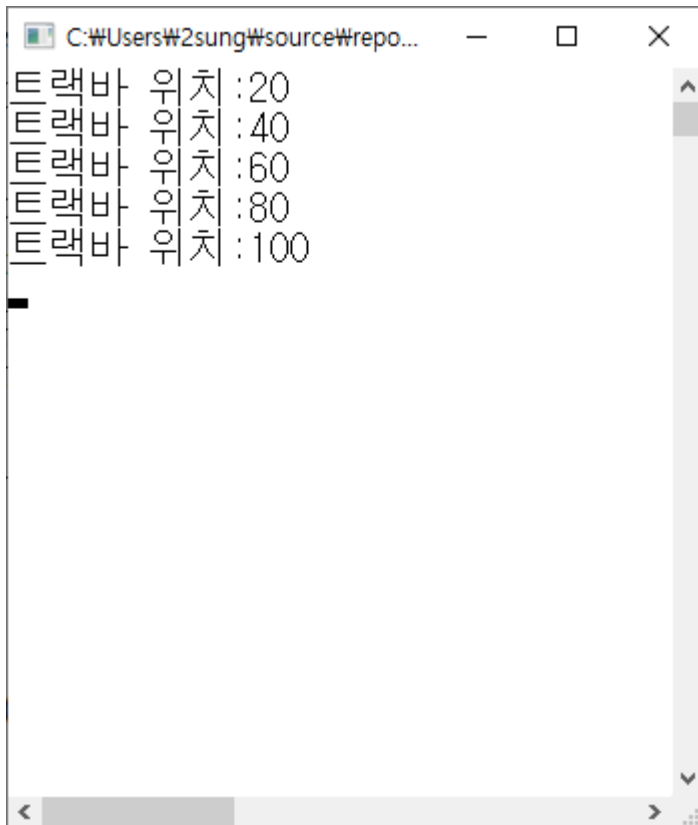
예제4: void포인터 이용



```
D:\Users\2sungryul\Dropbox>
트랙바 위치:0
트랙바 위치:1
트랙바 위치:2
트랙바 위치:3
트랙바 위치:4
```

실습과제1

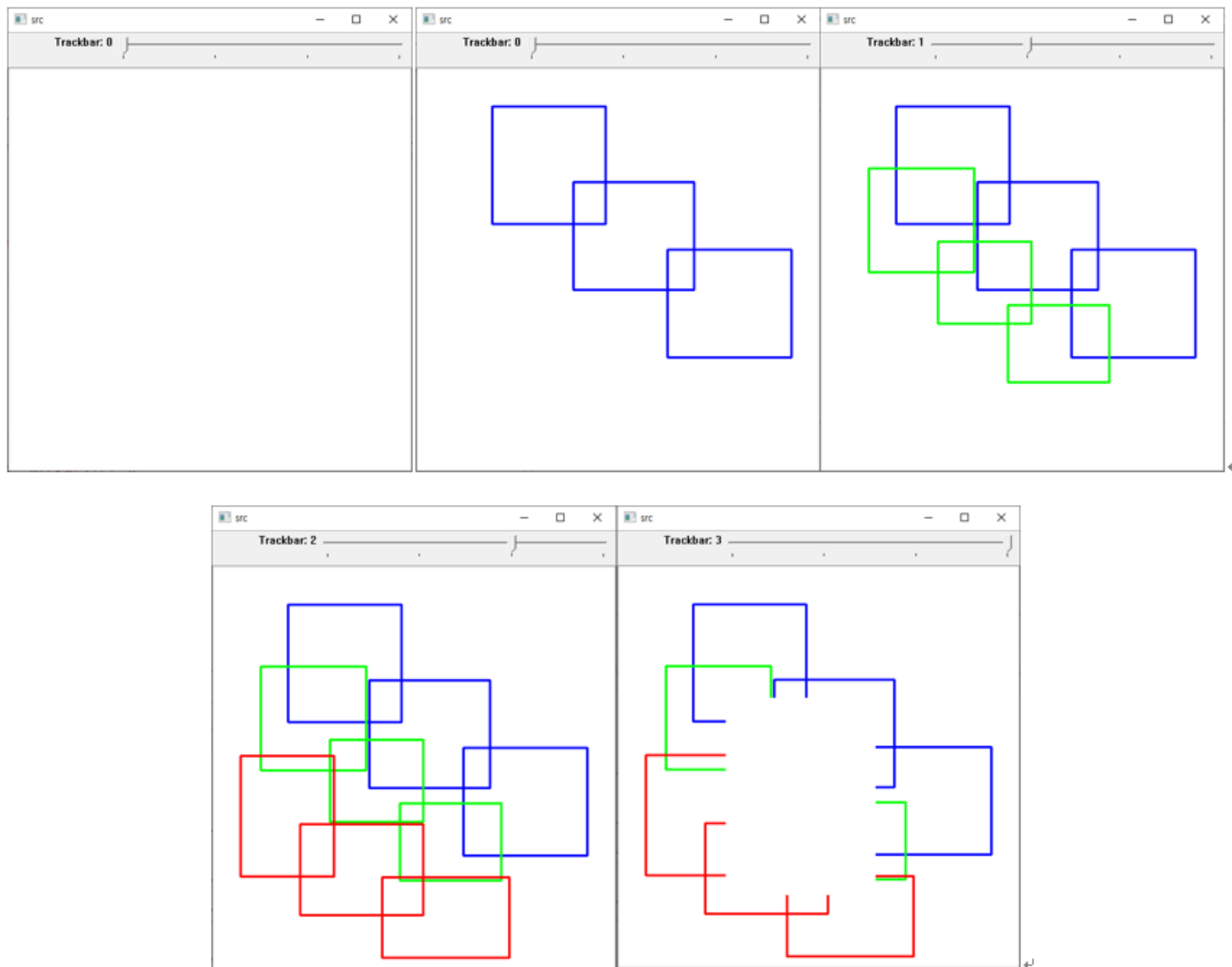
- 트랙바의 위치를 콘솔창에 출력하고 트랙바를 움직일때마다 트랙바의 위치값을 반지름으로 하는 원을 그리시오. 트랙바 위치의 최대값은 100으로 설정하라.



실습과제2

- 다음 지시대로 프로그램을 작성하시오.
- ① 초기화면은 1번그림처럼 폭 500, 높이 500픽셀의 3채널 컬러영상으로 설정하라. 트랙바는 0~3의 범위를 갖도록 하고 초기위치는 0으로 설정하라.
- ② 2번그림처럼 트랙바의 위치가 0일 때는 마우스로 드래그한 부분에 파란색 사각형이 그려지도록 하라.
- ③ 3번그림처럼 트랙바의 위치가 1일 때는 마우스로 드래그한 부분에 초록색 사각형이 그려지도록 하라.
- ④ 4번그림처럼 트랙바의 위치가 2일 때는 마우스로 드래그한 부분에 빨간색 사각형이 그려지도록 하라.
- ⑤ 5번그림처럼 트랙바의 위치가 3일 때는 마우스로 드래그한 부분이 지워지도록 하라.

실습과제2



과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.